

Microtask Batching

Q46

SVELTE 5 REACTIVITY MICROTASK PERFORMANCE

INTERVIEW QUESTION

Question. What is the microtask batching mechanism in Svelte, and how does it protect against redundant UI renders?

ANSWER.

The mechanism — what happens on every write

- **Two-step write:** every `** $state` assignment** (a line like `count = 5`) does two things in sequence — it stores the new value **immediately**, and it tells Svelte to schedule **one flush** later. A **flush** is Svelte's term for the moment it actually updates the screen.
- **Scheduled, not immediate:** that flush is queued on the **microtask queue** — a built-in JavaScript list of tiny tasks that the browser runs *after* your current function finishes, but *before* it paints anything. So if you write to the same variable ten times in a row, all ten writes land **before the flush ever runs**.
- **What the flush does:** Svelte looks at every `** $derived` value (a **computed value that depends on others**) whose inputs changed, **recomputes only those, and then writes the final values**** to the DOM. Intermediate steps are skipped entirely.

The protection — why renders are not redundant

- **One render per tick:** a loop calling `count += 1` **ten times** produces **one DOM update** at the end, not ten. The flush sees the final value of `count` and acts on that alone.
- **Multiple variables coalesce:** if a handler updates **several variables at**

once, each is **flagged as changed** (Svelte marks "this one's stale, re-check it"), but Svelte processes **all of those flags in a single flush** that runs once after the handler returns — never one flush per variable.

- **No wasted renders:** Svelte **never paints** an intermediate value that the user would never see, because the screen only updates after the flush.

The rule — what you should and should not do

- **Do not debounce:** never reach for **`**setTimeout`** or manual batching **to combine reactive writes** — the runtime already does it**, so adding your own delay just makes the UI feel laggy.
- ****Do await `tick()` when you need to read the DOM** after it has settled** (see Q47). `tick()` is Svelte's function that returns a Promise resolving **once the current flush finishes** — the **only** legitimate reason to wait for a flush.

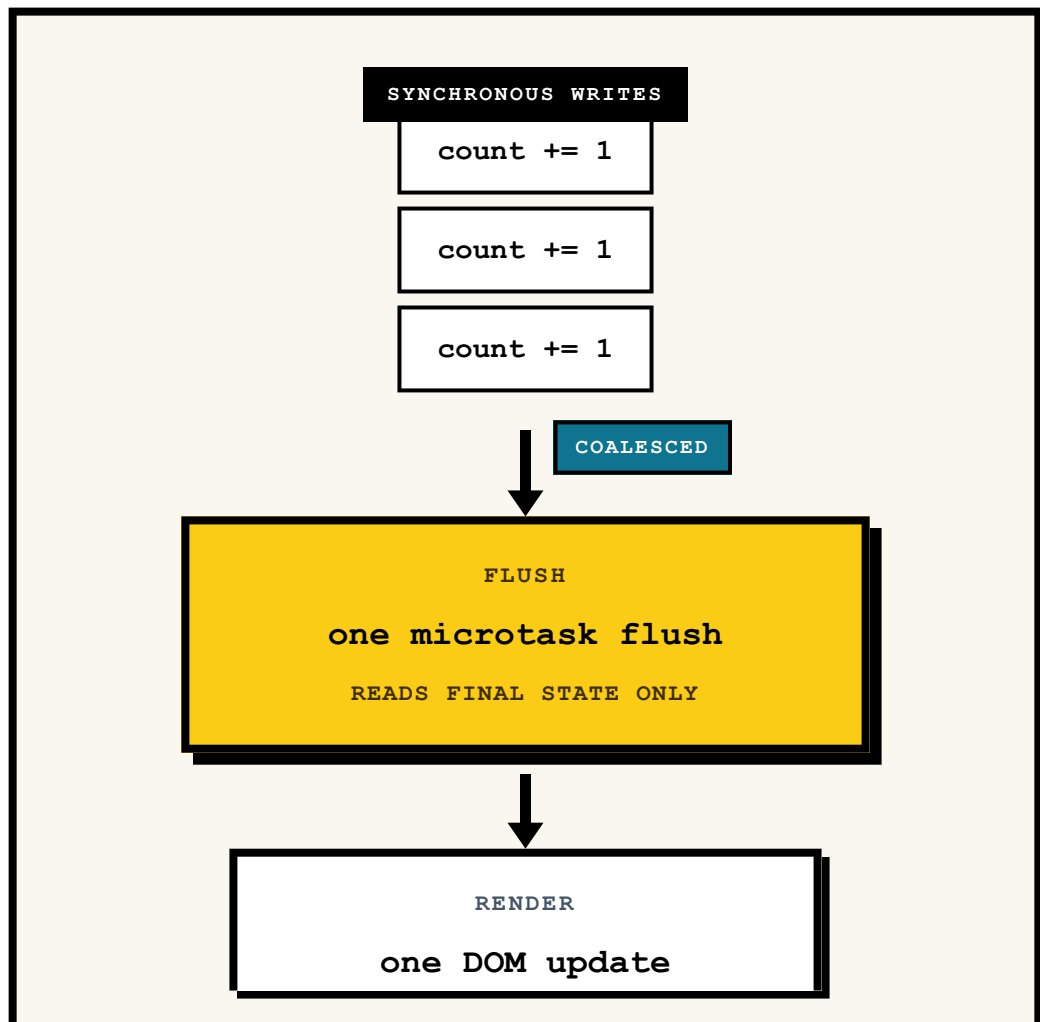
App.svelte

```
<script>
  let count = $state(0);
  let label = $state('ready');

  function burst() {
    // Ten synchronous writes — but Svelte schedules ONE
    flush.
    for (let i = 0; i < 10; i++) count += 1;
    label = `done at ${count}`;
    // When the microtask runs, count is 10 and label is 'done
    at 10'.
    // The DOM is updated exactly once with those final
    values.
  }
</script>

<button onclick={burst}>burst</button>
<p>{label}</p>
```

① **Summary.** Svelte schedules exactly one flush per microtask. Multiple synchronous writes to one or more signals coalesce into a single render of the final state, so intermediate values the user would never see are never written to the DOM. The runtime already batches — do not add manual debouncing on top.



count = 3